

Toolkit Script Library and Tutorial

Version 2.0 – Beginner & Modular Systems for Hyper-Casual Prototyping

All of these scripts also exist on prefabs as examples that you can play with <3

1. InputManager.cs

Purpose:

Handles player inputs (tap, click, swipe, pause) and sends them to other scripts.

Inspector setup:

- Usually placed on an empty GameObject named “**InputManager**” in the scene.
 - Make sure there’s only one per scene (Singleton pattern).
 - Other scripts (like TapAction, SwerveController) listen to it.
-

2. TapAction.cs

Purpose:

Responds to tap input — e.g., make the player jump (or flap wings f.x.).

How it works:

- In `Start()`, it subscribes to the `InputManager.OnTap` event.
- When tapped, it runs its own `HandleTap()` method.

Inspector setup:

- Attach to your Player or object that should react to taps.

- No manual linking needed if `InputManager` exists (it finds it automatically).
-

3. SwerveController.cs

Purpose:

Controls left/right movement based on A and D keyboard presses.

Modes available:

1. **Pan Mode:** Smooth left/right drag (like lane games).
2. **Turn Mode:** Turns 90° left/right on button push.

How to choose mode:

- In the Inspector, there's a dropdown or toggle (e.g. `swerveMode = Pan` or `swerveMode = Turn`).

Inspector setup:

- Attach to the Player.
 - No manual linking needed if `InputManager` exists (it finds it automatically).
 - Has tunable speed and turn angle values.
-



4. ForwardRunner

Purpose:

Moves the player forward constantly for endless or auto-runner styles.

Inspector setup:

- Attach to the Player.

- Contains public methods:
 - `StartRunning()`
 - `StopRunning()`
 - (These can be triggered from events, like `OnGameStart` or `OnGameLose`.)

Bonus:

Can sync with `GameStateManager` for start/stop logic.

5. DeathHandler.cs

Purpose:

Detects when the player collides with an obstacle.

Behavior:

- Fires `OnHittingObstacle` event immediately.
- Waits for a short delay (configurable) before notifying the `GameStateManager`.

Inspector setup:

- Add to Player prefab.
- Drag any visual/sound effects into the **OnHittingObstacle** UnityEvent.
- Set `eventDelay` (default 1 second).

Bonus:

- Can only detect obstacles that are gameobjects with colliders/triggers and have the tag: "Obstacle". Or you can use the Obstacle prefab.
-

23. ObstacleMover.cs

Purpose:

Makes obstacles move — either endlessly in one direction or back and forth — to create dynamic, challenging levels.

It's fully configurable in the **Inspector**, so no coding is needed!

How it works

The script automatically moves the obstacle based on the settings you choose:

- **MovementMode.Endless** → The obstacle moves forever in a single direction (like flying debris, rolling barrels, etc.).
- **MovementMode.PingPong** → The obstacle moves back and forth between two points (perfect for moving walls or swinging traps).

You can control:

- **Direction** (which way the object moves)
- **Speed** (how fast it moves)
- **Patrol Distance** (how far it goes before turning back — only for PingPong mode)

The script uses Unity's `Transform.Translate()` method to move the object and flips direction automatically when needed.

Inspector Setup

1. Create a new **Obstacle** GameObject (or use one already in your scene).
2. Add the **ObstacleMover.cs** script.
3. In the **Inspector**:
 - **Movement Mode**: Choose between `Endless` or `PingPong`.
 - **Move Direction**: Set a direction (for example, `(0, 0, 1)` moves forward, `(1, 0, 0)` moves right).
 - **Speed**: Set how fast the obstacle should move.

- **Patrol Distance:** (Only matters for PingPong) How far before it turns around.

 Tip for beginners:

If you don't see anything moving — make sure your speed is greater than 0, and your direction isn't (0, 0, 0)!

 Example Uses

- Moving barriers sliding left and right.
 - Floating platforms traveling back and forth.
 - Endless rolling hazards moving toward the player.
 - Conveyor belts that keep pushing forward.
-

7. Pickup.cs

Purpose:

Detects player collision with pickups and triggers effects or score increases.

Inspector setup:

- Add to coin, gem, or collectible objects.
 - Link effects (e.g. sparkle animation, sound, score manager) to the **OnPickup** event.
-



8. ScoreManager.cs

Purpose:

Tracks current score and highscore.

Inspector setup:

- Placed on a GameObject in the scene

- UI elements can subscribe to its events to do something when score is added
- Attach Score UI text element into the Inspector of the ScoreManager

Bonus:

- Includes a function to reset the highscore called `ResetScore()`. Can be hooked onto a button's event or GameStateManager event.
-

9. EffectTrigger.cs

Purpose:

Plays particle effects, animations, or other effects when triggered by an event.

Inspector setup:

- Add to an object that holds a particle system or Animator.
 - Connect it to a UnityEvent (e.g. `OnPickup`, `OnGameWin`).
 - Can be reused anywhere — think “plug-and-play reactions.”
-

10. GameStateManager.cs

Purpose:

Handles the overall state of the game (Playing, Lose, Win, Restart, Respawn, Next level).

Key events:

- `OnGameWin`
- `OnGameLose`
- `OnGameRespawnAtCheckpoint`

- `OnGameRestart`
- `OnGameNextLevel`

Inspector setup:

- Place on a GameObject called **GameStateManager**.
 - Other scripts can listen to these states.
-

11. ProceduralLevelGenerator.cs

Purpose:

Procedurally spawns level tiles (blocks or chunks of the same length) in a straight line (for endless or finite levels).

Features:

- Optional StartTile and EndTile (for finite levels).
- Weighted random tile selection.
- Tries to avoid overlapping tiles.
- Compatible with straight paths.

Inspector setup:

- Add to an empty GameObject named **LevelGenerator**.
 - Assign tile prefabs and counts.
 - Hit Play — it auto-generates your level.
-

12. TimerManager.cs

Purpose:

Handles timer logic for countdowns or elapsed time.

Inspector setup:

- Add to a GameObject in your scene.
 - Connect TimerDisplay to show time on UI.
 - Can be hooked into events like `OnGameLose`, `OnGameRestart`
-



13. TimerDisplay.cs

Purpose:

Shows time in minutes:seconds format on UI.

Works with:

- TimerManager (auto-syncs)

Inspector setup:

- Attach the Score UI Text element to the script in the inspector.
 - Displays time automatically.
 - No setup needed beyond linking the Text field.
-



14. SoundManager.cs

Purpose:

Handles background music and sound effects.

Inspector setup:

- Add to a persistent GameObject (often in the main menu). That way it will live in all scenes and keep playing f.x. the background music across levels.

- Use public methods `PlayMusic()`, `PlaySFX()` to trigger sounds.
 - Link events like `OnGameWin` → `PlaySFX(WinSound)`.
-



15. WinScreen.cs / LoseScreen.cs

Purpose:

Shows win or lose UI.

Inspector setup:

- Each screen can be its own Canvas prefab.
 - Initially disabled from view.
 - Can be hooked up to events in `GameStateManager.OnGameWin` or `.OnGameLose` using the functions `Show()` and `Hide()` in the Win/LoseScreens.
 - The event can then show or hide the screens.
-



16. SceneTransitionManager.cs

Purpose:

Handles switching between levels

Inspector setup:

- Place in every scene.
 - Includes functions like `LoadNextScene()`, `ReloadCurrentScene()`,
 - These can be linked to UI buttons or events.
-



17. PlayerRespawn.cs

Purpose:

Handles respawning the player at the last saved checkpoint — either instantly with a key press or automatically via the GameStateManager.

How it works:

- Saves the player's **starting position** (or last checkpoint reached).
- Moves the player back to that position when the **Respawn** key is pressed or when called by another script.
- Uses a static variable (`lastCheckpointPosition`) so the respawn point is always remembered.

Inspector setup:

- Attach this script to your **Player** GameObject.
- Optionally change the **Respawn Key** (default = `R`).
- Can be triggered by:
 - Pressing the key in play mode, or
 - A UnityEvent in `GameStateManager` (e.g., `OnGameRestartAtCheckpoint` or `OnGameLose` → `RespawnAtCheckpoint()`).

Example usage with GameStateManager:

1. Select your **GameManager** object.
 2. In the `GameStateManager` component, find the **OnGameRestartAtCheckpoint** or **OnGameLose** event.
 3. Click **+** to add a new listener.
 4. Drag your **Player** object into the field.
 5. From the dropdown, select:
`PlayerRespawn` → `RespawnAtCheckpoint()`
-

18. Checkpoint.cs

Purpose:

Marks positions in the level where the player should respawn after dying.

When the player touches a checkpoint, it updates the saved respawn position stored in `PlayerRespawn`.

How it works:

- Each checkpoint object has a trigger collider.
- When the player enters that trigger, the script updates the **static respawn position** in `PlayerRespawn`.
- The next time the player respawns, they'll start from this new position instead of the original spawn point.

Inspector setup:

- Add this script to any **checkpoint object** in your level (for example, a glowing pad, flag, or portal).
- Make sure the object's collider is set to **Is Trigger = true**.
- Tag your Player as "**Player**" so the script recognizes it.
- You can add optional visual feedback like a particle effect or light animation using a UnityEvent on `OnTriggerEnter`.

19. RaycastPlayerJump.cs

Purpose:

Lets the player **jump** when they press Space — but only if they're standing on the ground. It uses a **raycast** to check if the player is grounded before applying an upward force.

How it works:

- Casts an invisible **ray** straight down from the player's position every frame.

- If that ray hits something on the “Ground” layer within a set distance, the player is considered grounded.
- When the player presses **Space** and is grounded, a jump force is applied to their Rigidbody.

Inspector setup:

- Attach this script to your **Player** object.
- Make sure the Player has a **Rigidbody** component.
- Set up these fields:
 - **Jump Force** – how strong the jump is (default 5).
 - **Ground Check Distance** – how far down to check for the ground (default 1.1).
 - **Ground Layer** – assign your “Ground” layer here (create one if it doesn’t exist).

Example usage in scene:

1. Select your **Player** GameObject.
2. Add the **RaycastPlayerJump** script.
3. Make sure your ground objects (floor, platforms, etc.) are on a **layer** called “Ground”.
4. In the script’s **Ground Layer** field, select “Ground”.
5. Hit **Play** — the player can now jump when standing on the ground!

20. PauseMenuManager.cs

Purpose:

Handles pausing and resuming the game, as well as showing and hiding the pause menu UI. It listens for a “pause” input event (from the **InputManager**) and freezes time when activated.

How it works:

- Subscribes to the `InputManager.OnPause` event at startup.
- When the pause event is triggered (e.g., player presses Escape or a pause button):
 - The game time stops (`Time.timeScale = 0`).
 - The pause menu Canvas becomes visible.
- When unpaused, time resumes and the menu hides again.
- If `InputManager` isn't found, the script still works if you call `PauseGame()` or `ResumeGame()` manually (for events on UI buttons).

Inspector setup:

- Create a new **Pause Menu Canvas** (UI → Canvas → "PauseMenu").
- Add your pause buttons (Resume, Main Menu, Quit, etc.) inside this Canvas.
- Add the `PauseMenuManager` script to the same GameObject as the Canvas.
- Make sure the Canvas has a **Canvas** component and is set to **Screen Space - Overlay**.
- No manual event wiring needed if `InputManager` is in the scene — it connects automatically.

Example usage in scene:

1. Create your Pause Menu UI (with Resume/Quit buttons).
2. Add `PauseMenuManager.cs` to the **PauseMenu Canvas** object.
3. (Optional) Add `Button` components and hook them up like this:
 - **Resume Button** → `PauseMenuManager.ResumeGame()`
 - **Quit Button** → `SceneManager.LoadScene()` or `Application.Quit()`

4. Play the scene — press your pause key (set in `InputManager`), and the menu appears!
-

21. Goal.cs

Purpose:

Marks the **end goal** of a level.

When the player enters its trigger area, the game registers a **win** through the `GameStateManager`.

How it works:

- Uses a **trigger collider** to detect when the player enters the goal area.
- Checks if the object that entered is tagged as “Player.”
- If so, it calls `GameStateManager.Instance.WinGame()` to activate the win state.
- This can then show a Win Screen, stop player movement, play effects, and more — all handled by the toolkit’s existing systems.

Inspector setup:

- Create an empty `GameObject` where you want the goal to be.
- Add a **Collider** component (like a `BoxCollider`).
- Enable **Is Trigger** on the collider.
- Add the **Goal.cs** script.
- (Optional) Add a visible object or green area so players can see where the goal is.
- Make sure your **Player** `GameObject` has the tag “**Player**.”

Example usage in scene:

1. Create a new empty `GameObject` named **Goal**.

2. Add a **BoxCollider**, check **Is Trigger**.
3. Add the `Goal1.cs` script.
4. Position it where you want the player to reach the end of the level.
5. (Optional) Add a visible object (like a green cube or glowing portal) as a child so players can see the goal zone.